# GAN và diffusion

Từ noise đến mẫu có cấu trúc

Phase 5 · Bài 5.8

# Một ảnh đẹp chưa đủ

Model cần chất lượng và đa dạng.

Noise cung cấp sự biến thiên.

# Seed và “cùng điều kiện”

Seed cố định dãy số ngẫu nhiên → cố định noise.

Tái tạo được khi cùng model, sampler, phiên bản thư viện, phần cứng.

Đổi một thứ, ảnh có thể khác dù seed giữ nguyên.

HỎI NHANH

# Một đội giữ nguyên seed 7, prompt và model, chỉ đổi sampler, chạy trên cùng một máy. Ảnh ra khác hẳn. Vì sao?

A Seed chỉ cố định các số ngẫu nhiên, còn sampler đổi cách đi từ noise tới ảnh

B Đổi sampler làm seed bị bỏ qua, nên noise ban đầu được lấy lại ngẫu nhiên

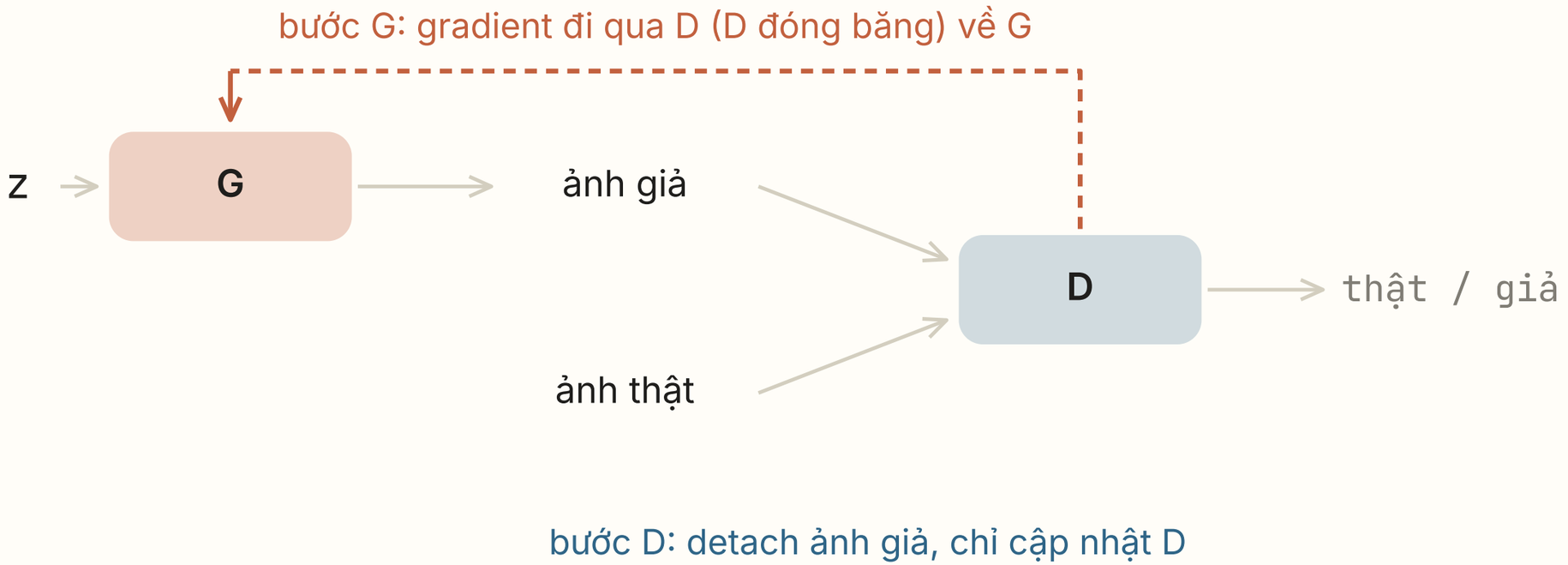
C Cùng seed thì phải cùng ảnh, nên ảnh khác nghĩa là máy đang tính sai

D Mỗi sampler đi kèm trọng số riêng, nên thực chất model đã bị đổi

01

# Hai mạng của GAN

# Hai bước cập nhật




GAN: bước D học phân biệt ảnh thật với ảnh giả đã detach; bước G giữ D đóng băng nhưng cho gradient đi qua D về generator.

# Loss thực hành

: nhãn 1 cho thật, 0 cho giả.

Non-saturating G: .

: loss .

# Vì sao non-saturating

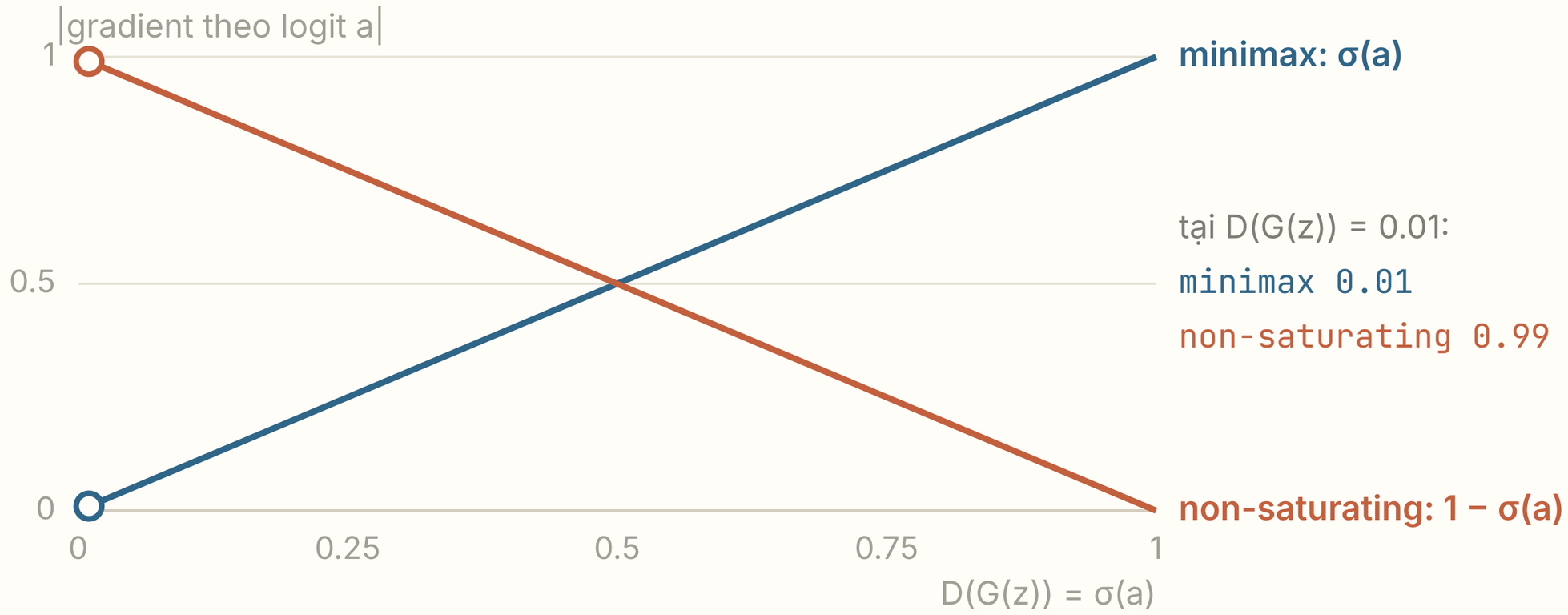
, với là logit của fake.

Minimax:

Non-saturating:

Đầu train : minimax gần 0, non-saturating gần 1.

# Gradient khi D thắng dễ




Độ lớn gradient của loss generator theo logit của ảnh giả: khi D(G(z)) = 0.01, minimax chỉ còn 0.01 còn non-saturating còn 0.99.

HỎI NHANH

Đầu train, D dễ nhận ra ảnh giả: . Tính theo logit của ảnh giả, độ lớn gradient của , tức loss minimax, và của , tức loss non-saturating, lần lượt là bao nhiêu?

A và

B và

C và

D Bằng nhau, vì hai loss cùng đẩy lên

# Bước D tách fake

Đoạn trích: `real_logits=D(real)` và `fake=G(noise)` đã tính; `criterion` nhận logits và target thật/giả.

```
optimizer_d.zero_grad()
fake_logits = discriminator(fake.detach())
loss_d = criterion(real_logits, torch.ones_like(real_logits))
loss_d += criterion(fake_logits, torch.zeros_like(fake_logits))
loss_d.backward()
optimizer_d.step()
```

# Bước G truyền gradient qua D

Code của bài dùng hai module linear; `noise` là batch đầu vào của G. Giữ tham số D cố định, nhưng vẫn ghi graph từ output D về input G.

```
for parameter in discriminator.parameters():
    parameter.requires_grad_(False)
optimizer_g.zero_grad()
fake = generator(noise)
fake_logits = discriminator(fake)
loss_g = criterion(fake_logits, torch.ones_like(fake_logits))
loss_g.backward()
optimizer_g.step()
for parameter in discriminator.parameters():
    parameter.requires_grad_(True)
```

# Kiểm tham số ở hai bước

```
print("D unchanged during G step:", torch.equal(d_before_g, discriminator.weight))
print("G changed during G step:", not torch.equal(g_before_g, generator.weight))
```

KẾT QUẢ

```
D unchanged during G step: True
G changed during G step: True
```

# Đọc hai loss

D loss 1.4571, gần : D mới chưa phân biệt được.

G loss 1.0656: trung bình nhân .

D còn nghiêng về “giả”: G còn tín hiệu để học.

HỎI NHANH

## Xóa `.detach()` ở bước D rồi chạy lại. Dòng kiểm vẫn in `G unchanged during D step: True`. Ta kết luận được gì?

A Bỏ `detach()` là an toàn, vì phép kiểm đã in True

B Trọng số G không đổi, nhưng `.grad` của G đã bị ghi

C Phép kiểm sai, trọng số G thật ra đã bị cập nhật

D Có detach hay không thì gradient bước D cũng không tới được G

HỎI NHANH

## Ở bước G, ta bọc dòng `fake_logits = discriminator(fake)` trong `torch.no_grad()`, còn `fake = generator(noise)` vẫn ở ngoài. Chuyện gì xảy ra?

A G vẫn học, vì `fake` được tạo ngoài khối `no_grad`

B Code chạy được nhưng chỉ cập nhật D, vì D nằm trong khối

C `loss_g.backward()` báo lỗi, vì loss không còn nối tới G

D G học chậm hơn, vì gradient qua D bị thu nhỏ lại

# Mode collapse

Sinh vài kiểu giống nhau.

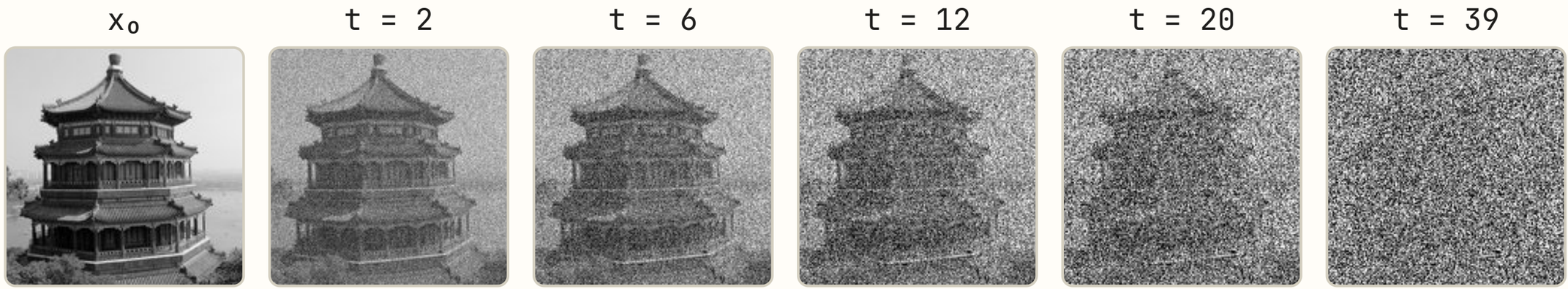
Loss không thay được việc đánh giá chất lượng và đa dạng.

Xem nhiều seed, không chọn riêng ảnh đẹp.

02

# Nhiều mức noise của diffusion

# Thêm nhiễu và chiều sinh




forward: thêm nhiễu theo lịch β của bài; sinh ảnh là học đi ngược chiều này

Một ảnh thật đi qua forward diffusion với đúng lịch nhiễu của bài (40 mức, β từ 0.01 đến 0.15): ᾱ giảm dần, đến mức cuối ảnh gần như chỉ còn nhiễu.

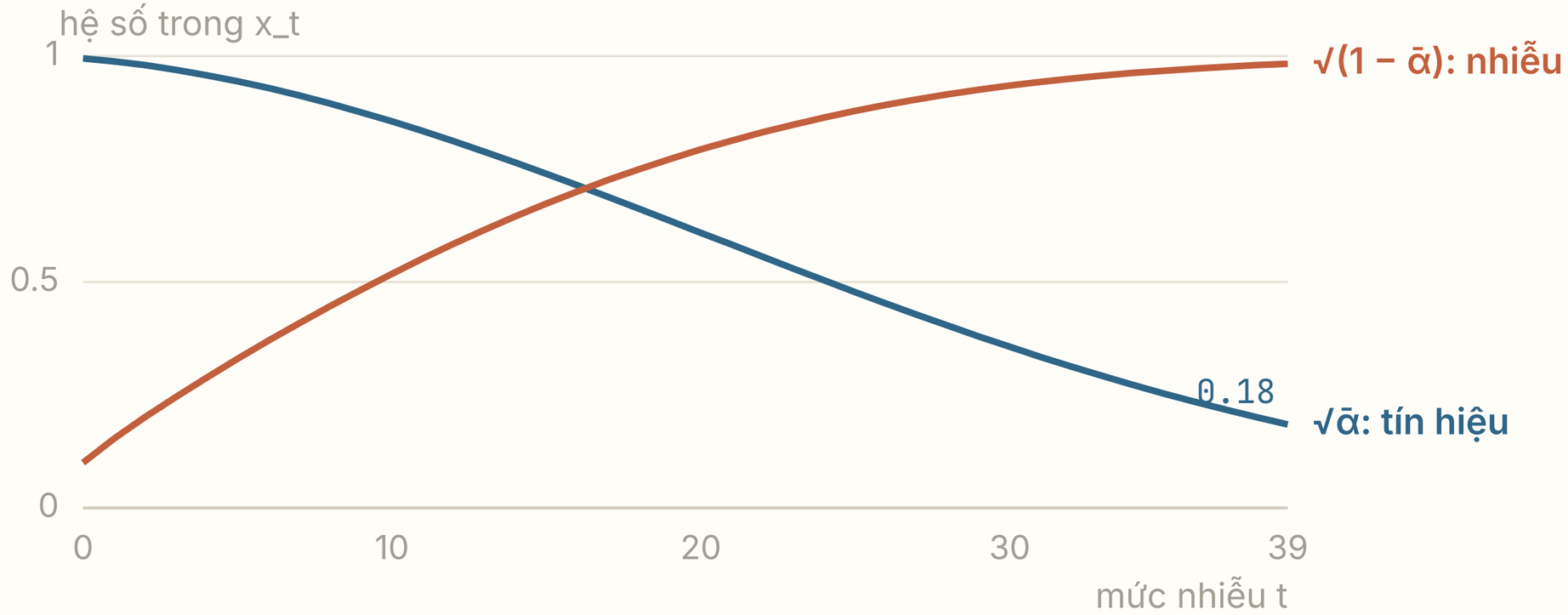
# Lấy mẫu ở mức

$\sqrt{\quad}$ $\sqrt{\quad}$

là mức noise, không phải epoch; là noise Gaussian chuẩn đã rút.

: phần tín hiệu còn lại theo lịch nhiễu.

# Tín hiệu và nhiễu theo mức




Hai hệ số của x_t = √ᾱ·x₀ + √(1 − ᾱ)·ε theo lịch nhiễu của bài: hệ số tín hiệu giảm từ gần 1 xuống 0.18 ở mức cuối, hệ số nhiễu tăng lên gần 1.

# Tính tay

Target khi train noise predictor là , không phải 2.

# Khôi phục bằng noise đã biết

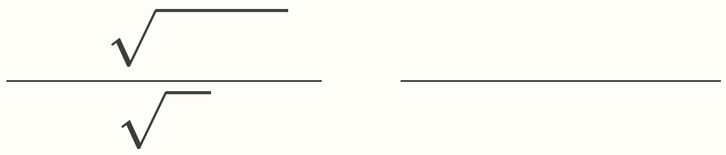
```
print("noisy:", round(noisy, 4))
print("reconstruction with known noise:", round(estimated_clean, 4))
```

KẾT QUẢ

```
noisy: 1.9
reconstruction with known noise: 2.0
```

# Một kiểm đại số

Khôi phục 2 dùng noise thật.



Lúc sinh: chưa biết ảnh sạch.

Noise predictor phải ước lượng.

Đây chưa phải sampler hoàn chỉnh.

HỎI NHANH

Trong ví dụ tính tay có , , và . Một mẫu train của noise predictor gồm đầu vào và target nào?

A Đầu vào , target

B Đầu vào , bỏ , target

C Đầu vào , target

D Đầu vào , target

# 03

# Học và điều khiển

# Noise predictor nhỏ học thật

KẾT QUẢ

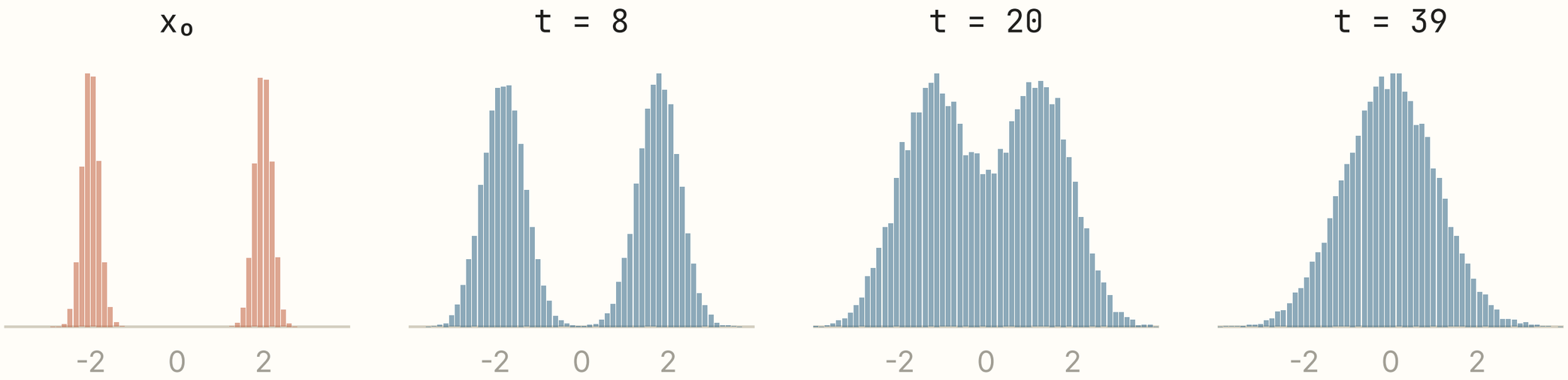
```
noise MSE before/after: 0.9092 0.3434
zero predictor MSE: 0.9815
last alpha_bar: 0.0342
```

MLP nhận và mức (chia 39), đoán bằng MSE.

0.3434, thấp hơn nhiều so với 0.9815 của baseline luôn đoán 0: model đã học.

$\sqrt{\phantom{x}}$ : mức cuối gần noise chuẩn.

# Hai cụm nhòe thành một




hai cụm quanh ±2 nhòe dần thành một phân phối chuẩn

Dữ liệu một chiều của bài (hai cụm quanh −2 và +2) sau forward diffusion ở các mức nhiễu: hai đỉnh nhòe dần và gộp thành phân phối gần chuẩn ở mức cuối.

# MSE không về 0

Cùng đến từ nhiều cặp .

Nhiễu thấp: noise nhỏ hơn độ lệch của quanh .

Mức giữa: không biết cụm hay .

HỎI NHANH

## Ở một mức có , noise predictor đoán từ dễ hay khó?

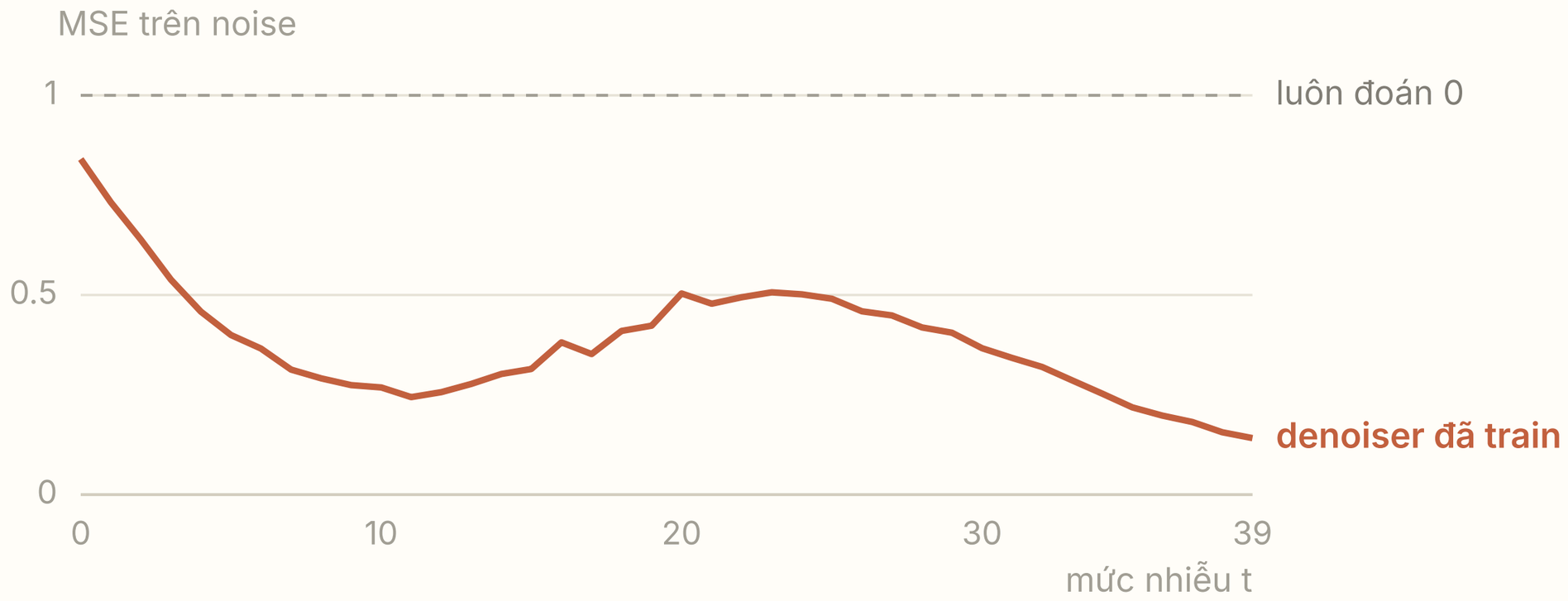
A Dễ, vì , gần như đọc thẳng được noise

B Khó nhất, vì tín hiệu gần mất hết, không còn gì để dựa vào

C Không xác định, vì phải biết mới tính được

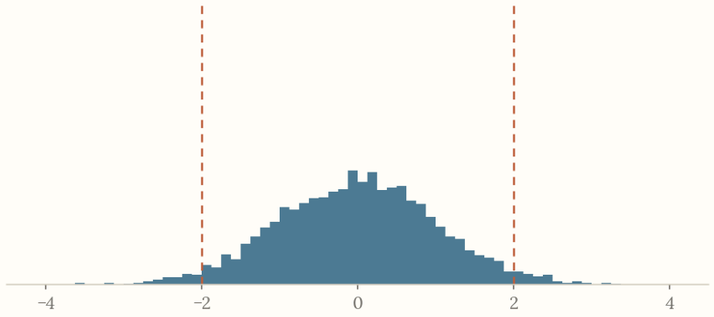
D Khó, vì noise lớn nên MSE ở mức này luôn lớn nhất

# MSE theo mức nhiễu

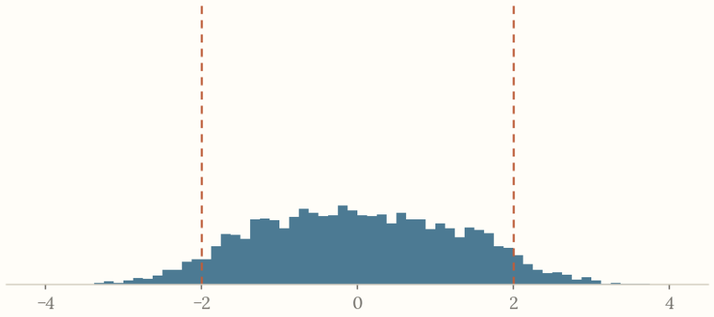



MSE của noise predictor đã train (code bài) theo từng mức nhiễu: khó ở mức thấp, nơi noise nhỏ hơn độ lệch của $x_0$, và ở mức giữa, nơi không biết x_t đến từ cụm nào; dễ dần ở mức cao, nơi x_t gần bằng chính noise.

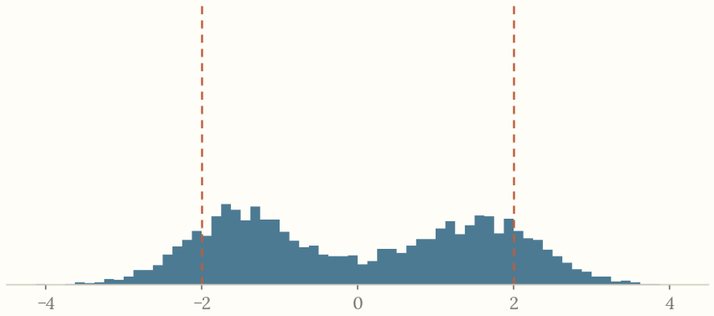
# Sampler đi từ noise về hai cụm



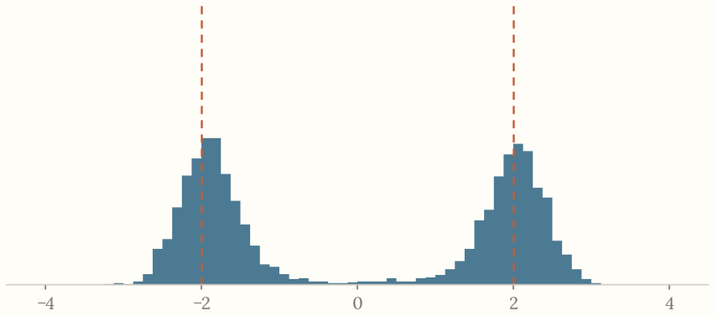

mức 40: điểm xuất phát lấy từ N(0, 1)




sau 12 bước khử nhiễu, còn mức 28




sau 24 bước khử nhiễu, còn mức 16




mức 0: hai cụm quanh −2 và +2, như dữ liệu train

Sampler DDPM dùng noise predictor đã train của bài: 4000 điểm từ noise chuẩn dần tách về hai cụm ±2.

# Sampler: nhiều lần gọi mạng

Bắt đầu , đi về .

Hệ số reverse phải khớp lịch nhiễu lúc train.

50 bước là 50 lần forward nối tiếp.

GAN: một lần.

# Latent và prompt

Prompt → embedding → denoiser (cross-attention).

Latent diffusion: encoder nén, khử nhiễu trên latent, decoder ra pixel.

Stable Diffusion 1.x: latent cho ảnh .

HỎI NHANH

Dùng latent thay cho ảnh . Mỗi lần gọi denoiser xử lý ít hơn bao nhiêu lần số giá trị?

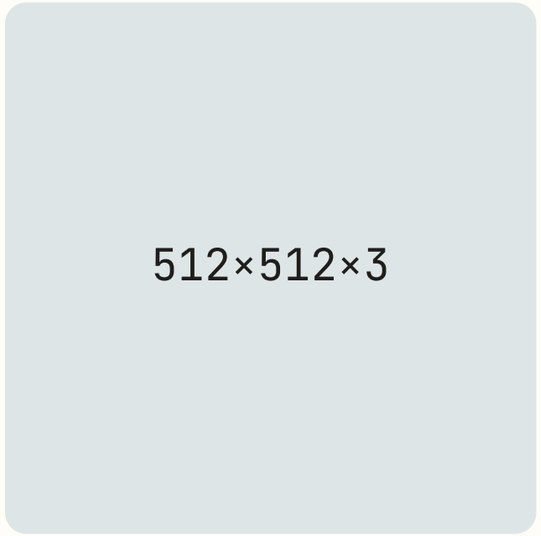
A 8 lần

B 48 lần

C 64 lần

D 192 lần

# Ảnh và latent




mỗi lần gọi denoiser

**ít hơn 48 lần**

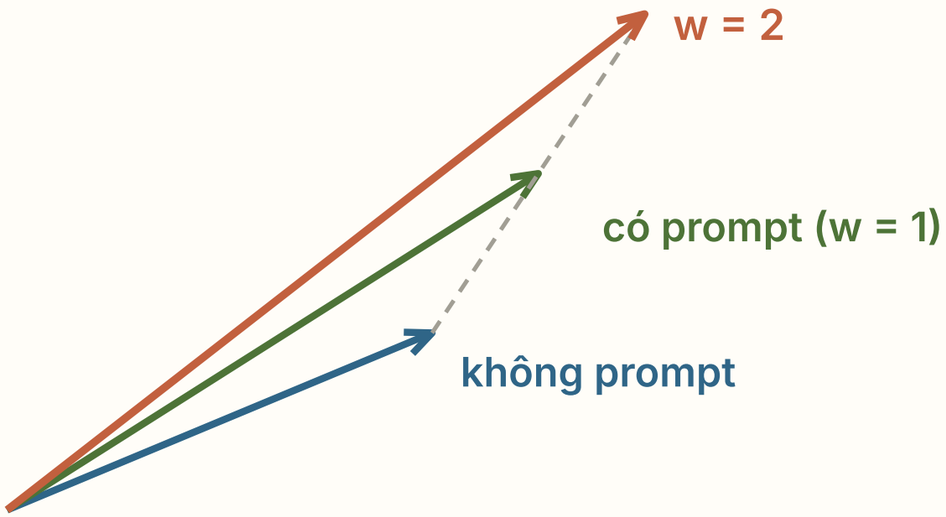
Latent diffusion khử nhiễu trên latent 64×64×4 = 16 384 số thay vì ảnh 512×512×3 = 786 432 số, ít hơn 48 lần mỗi lần gọi denoiser.

# Classifier-free guidance

lớn: bám prompt hơn, kém đa dạng, mỗi bước gọi hai lần.

Ghi bước, sampler, kích thước,  , seed, prompt.

# Guidance đẩy xa bản không prompt




$$\hat{\varepsilon} = \varepsilon_\varnothing + w\,(\varepsilon_c - \varepsilon_\varnothing)$$

w lớn: bám prompt hơn,
kém đa dạng, tốn gấp đôi

Classifier-free guidance: lấy dự đoán không prompt rồi đi tiếp theo hướng chênh lệch sang dự đoán có prompt, w lần; w = 1 là dự đoán có điều kiện thường.

# Recipe diffusion cục bộ

`local_assets.py --task diffusion`: kiểm diffusers, thư mục pipeline, prompt, ảnh ra.

Không `--run`: chỉ kiểm rồi dừng.

`--run`: offline, CPU float32, một ảnh , 10 bước, seed 5.

Xác nhận đường nạp; so chất lượng cần cả bộ prompt và seed.

# Bẫy: một ảnh chọn đẹp

Cố định prompt, seed và cấu hình; sinh nhiều mẫu.

Đọc chất lượng, đa dạng, mức đúng điều kiện.

FID cần hàng nghìn mẫu, cách tính nhất quán.

HỎI NHANH

## Ta so hai cấu hình sampler. Cấu hình A cho 4 ảnh đẹp nhất chọn từ 200 lần chạy. Cấu hình B cho 4 ảnh của seed 0 đến 3. A trông đẹp hơn. So thế nào mới công bằng?

A Chọn A, vì ảnh đẹp nhất cho thấy giới hạn trên của cấu hình

B Lấy ảnh đẹp nhất trong 200 lần chạy của cả hai cấu hình rồi so

C Dùng cùng prompt và seed, sinh nhiều mẫu, đo cả chất lượng lẫn đa dạng

D Bỏ ảnh đi, so loss train của hai cấu hình

# Khi đi thi

- Đọc đề: chấm ảnh, classifier hay câu tính tay; luật asset.
- Thời gian số ảnh số bước một lần gọi; CFG gấp đôi.
- So cấu hình: cùng prompt và seed, mỗi lần đổi một thứ.
- Ảnh sinh chỉ vào train; validation ảnh thật.
- Câu lý thuyết: viết $\sqrt{}$ $\sqrt{\phantom{x}}$ trước.

04

# Luyện đề

LUYỆN ĐỀ 1

Forward diffusion hai bước: $\sqrt{\quad}$ $\sqrt{\quad}$ và $\sqrt{\quad}$ $\sqrt{\quad}$, với độc lập, , . Phân phối của khi biết có mean và phương sai bao nhiêu?

A Mean $\sqrt{\quad}$, phương sai

B Mean $\sqrt{\quad}$, phương sai

C Mean , phương sai

D Mean , phương sai

PHỎNG THEO USA-NA-AIO 2026 · Vòng 2 · Ngày 2 · Part 3.1.1

LUYỆN ĐỀ 2

# Đề cho 30 phút để sinh 600 ảnh trên một GPU. Generator của GAN tạo một ảnh trong 15 ms. Pipeline diffusion dùng 50 bước, mỗi bước gọi U-Net một lần mất 40 ms, không dùng CFG. Diffusion mất bao lâu để sinh xong, và vì sao chậm hơn GAN?

A Khoảng 20 phút, vì mỗi ảnh cần 50 lần gọi mạng nối tiếp nhau

B Khoảng 20 phút, vì model diffusion có nhiều tham số hơn

C Khoảng 24 giây, vì mỗi ảnh chỉ cần một lần gọi U-Net

D Khoảng 24 giây, vì các bước khử nhiễu chạy song song

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 255

LUYỆN ĐỀ 3

Ở một bước sinh, denoiser cho khi không có prompt, và khi có prompt. Với classifier-free guidance , noise dùng cho bước này là bao nhiêu? lớn có tác dụng gì?

A , và sampling nhanh hơn vì cần ít bước hơn

B , và ảnh bám prompt hơn, đổi lại kém đa dạng

C , và ảnh bám prompt hơn, đổi lại kém đa dạng

D , và sampling nhanh hơn vì cần ít bước hơn

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 254

# Chốt ý

GAN: cập nhật luân phiên đúng gradient.

Diffusion: học đoán noise bằng MSE, sinh bằng nhiều bước reverse.

Đánh giá bằng nhiều mẫu với cùng prompt và seed.

# Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; xóa `.detach()` ở bước D, đoán trước ba dòng kiểm tham số rồi xem `.grad` của G.